

Compétences DWWM couvertes par MoniteurConnect

Titre professionnel **Développeur web et web mobile** (niveau 5), référentiel **REAC RNCP37674**. Libellés des blocs et compétences vérifiés le 2026-07-10 sur <https://www.francecompetences.fr/recherche/rncp/37674/> (titre valide du 01/09/2023 au 01/09/2028).

Pour chaque compétence : la réalisation concrète dans le projet, puis les preuves à montrer (fichier, test, document, moment de la démonstration).

Bloc 1 — « Développer la partie front-end d'une application web ou web mobile sécurisée »

1. Installer et configurer son environnement de travail en fonction du projet web ou web mobile

Réalisation : environnement Node.js reproductible, configuration par variables d'environnement avec refus de démarrer si un secret manque, migrations de schéma versionnées, méthode de travail documentée.

Preuve	Où
Modèle d'environnement (dev/prod distingués)	<code>.env.example</code>
Fail-fast sur <code>SESSION_SECRET</code> au démarrage	<code>src/server.js</code>
Scripts outillés : dev, test, seed, purge, admin	<code>package.json</code>
13 migrations Prisma versionnées	<code>prisma/migrations/</code>
Méthode TDD et conventions du dépôt	<code>../.. /AGENTS.md</code>

2. Maquetter des interfaces utilisateur web ou web mobile

Réalisation : 11 écrans maquetés en HTML basse fidélité avant le développement (23 juin), exports PNG et planche SVG, puis comparaison argumentée maquette/réalisation pour le jury.

Preuve	Où
Maquettes navigables v1 (originaux préservés)	<code>../historique/2026-06/wireframes/</code>
Exports PNG des 11 écrans	<code>../historique/2026-06/spec-assets/wf-*.png</code>
Comparaison v1 / application finale	<code>comparaison-maquettes.md</code>

3. Réaliser des interfaces utilisateur statiques web ou web mobile

Réalisation : rendu serveur Twig (échappement automatique), feuille de style unique aux composants réutilisés (cartes, badges, tuiles, messages), sémantique et accessibilité de base.

Preuve	Où
30+ vues Twig par domaine	views/ (layout : <code>views/layouts/base.twig</code>)
Styles : palette, grilles <code>auto-fit</code> , composants	<code>public/css/style.css</code>
Accessibilité : lien d'évitement, <code>:focus-visible</code> , <code>aria-live</code>	<code>views/layouts/base.twig</code> , <code>public/css/style.css</code>
Démo : parcours public complet	captures captures/

4. Développer la partie dynamique des interfaces utilisateur web ou web mobile

Réalisation : JavaScript navigateur dédié par fonctionnalité, sous CSP stricte (`script-src 'self'` , zéro script inline) : cartes Leaflet, pad de signature avec import d'image, graphiques SVG construits en DOM, autocomplétion débouncée, vérification SIRET en direct.

Preuve	Où
Carte détail + carte de recherche	<code>public/js/listing-map.js</code> , <code>public/js/listings-map.js</code>
Pad de signature (dessin + import PNG/JPEG)	<code>public/js/signature-pad.js</code>
Graphiques SVG sans bibliothèque, anti-XSS	<code>public/js/dashboard-charts.js</code> + <code>test/lot-h.cjs</code> (« aucune insertion HTML »)
Autocomplétion d'adresse débouncée	<code>public/js/adresse-autocomplete.js</code> + <code>test/lot-l.cjs</code>
CSP stricte vérifiée par test	<code>test/smoke.cjs</code> (« en-tête Content-Security-Policy »)

Bloc 2 — « Développer la partie back-end d'une application web ou web mobile sécurisée »

5. Mettre en place une base de données relationnelle

Réalisation : schéma Prisma de 8 modèles, contraintes d'intégrité (clés étrangères, unicités simples et composées, relation 1-1, cascades), évolution par migrations. Le code Prisma est portable vers PostgreSQL, avec un historique de migrations PostgreSQL distinct à préparer et tester.

Preuve	Où
Schéma commenté (8 modèles)	<code>prisma/schema.prisma</code>
Diagramme v2 + lecture guidée	base-de-donnees.md
Évolution versionnée	<code>prisma/migrations/</code> (13 migrations)
Contraintes vérifiées par tests	<code>test/correctifs.cjs</code> (P2002), <code>test/lot-g.cjs</code> (1-1), <code>test/smoke.cjs</code> (cascades)

6. Développer des composants d'accès aux données SQL et NoSQL

Réalisation : couche services dédiée (un module par domaine) au-dessus de Prisma, requêtes scopées par `schoolId` (isolation entre écoles), pagination mutualisée, recherche insensible à la casse portable SQLite/PostgreSQL.

Honnêteté sur le NoSQL : l'application n'utilise pas de base NoSQL en production — les données (écoles, annonces, candidatures, contrats) sont fortement relationnelles et exigent des contraintes d'intégrité. Les structures non relationnelles assumées du projet sont les **caches mémoire clé-valeur** des services externes (géocodage, SIRET, adresse) et le **JSON** des sessions persistées ; ce choix est argumenté tel quel devant le jury.

Preuve	Où
Services d'accès aux données	<code>src/services/</code> (<code>listingService</code> , <code>applicationService</code> , <code>statsService</code> ...)
Isolation <code>schoolId</code> testée (404 croisé)	<code>test/smoke.cjs</code> (« École B ne peut pas... »)
Pagination commune	<code>test/lot-a.cjs</code> , <code>test/ameliorations-v2.cjs</code>
Caches clé-valeur TTL	<code>src/services/adresse.js</code> (10 min), <code>src/services/siret.js</code> (1 h), <code>src/services/geocoder.js</code> (24 h)

7. Développer des composants métier coté serveur

Réalisation : logique métier en services testés — workflow de contrat et signature électronique (validation d'image par magic bytes, PDF, horodatages, empreintes SHA-256, invalidation à la ré-édition), purge RGPD planifiée et journalisée, alertes en double opt-in, statistiques avec bucketing hebdomadaire.

Preuve	Où
Signature : image, PDF, empreintes	<code>src/services/signatureImage.js</code> , <code>src/services/contractPdf.js</code> + <code>test/lot-g.cjs</code> (49 assertions)
Purge RGPD (délais, journal)	<code>src/services/purgeService.js</code> + <code>test/lot-j.cjs</code>
Alertes double opt-in, jamais bloquantes	<code>src/services/alertService.js</code> + <code>test/lot-i.cjs</code>
Statistiques (séries, entonnoir, taux)	<code>src/services/statsService.js</code> + <code>test/lot-h.cjs</code>
Focus démo recommandé : la signature (6 min)	audit-certification-dwww.md , section 3

8. Documenter le déploiement d'une application dynamique web ou web mobile

Réalisation — partiellement couverte, assumé : la configuration d'environnement est documentée (`.env.example` , distinction dev/prod dans `src/app.js` : cookie `secure` et `trust proxy` en production) et la bascule SQLite → PostgreSQL est décrite dans le schéma et `base-de-donnees.md` . La procédure de déploiement complète (hébergement, HTTPS, SMTP, sauvegarde/restauration) est le prochain chantier documentaire — feuille de route de l'[audit](#). Devant le jury : présenter ce qui existe et le plan, sans survendre.

Preuve	Où
Variables et secrets documentés	<code>.env.example</code> , <code>README.md</code>

Preuve	Où
Comportements spécifiques production	<code>src/app.js</code> (proxy, cookies <code>secure</code>)
Bascule base de données	<code>prisma/schema.prisma</code> (commentaire datasource), <code>base-de-donnees.md</code>

Synthèse pour la soutenance

Les compétences 1 à 7 s'appuient sur du code livré et testé (448 assertions) ; la compétence 8 s'appuie sur une documentation partielle et un plan explicite. Le déroulé de présentation qui couvre ces compétences minute par minute est dans l'[audit](#), [section 4](#).